

Informed Search, cont'd

1 A* Search

Let $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of reaching node n from the start state and $h(n)$ is an heuristic estimate of the distance from node n to the nearest goal. The main idea of A* search is to expand the lowest-cost node on the fringe, according to the evaluation function f . Like best- g and best- h searches, A* is a special case of the best-first search algorithm (see Table 1).

A* SEARCH	
Inputs	search problem $\langle X, S, G, \mathcal{T}, c \rangle$ heuristic function h
Output	(path to) optimal goal node
Initialize	$O = S$ is the list of open nodes
<pre> while (O is not empty) do 1. delete node $n \in O$ s.t. $f(n)$ is minimal 2. if $n \in G$, return (path to) n 3. for all $m \in \mathcal{T}(n)$ (a) compute $h(m)$ (b) $g(m) = g(n) + c(n, m)$ (c) $f(m) = g(m) + h(m)$ (d) insert m into O with priority $f(m)$ fail </pre>	

Table 1: A* Search.

1.1 Optimality

A* search is optimal, assuming the heuristic function h is admissible.

Definition An heuristic function h is **admissible** iff $h(n) \leq h^*(n)$, for all nodes n .

Assumption For all goal nodes m^* , by admissibility, $h(m^*) \leq h^*(m^*) = 0$, so $h(m^*) \leq 0$. Furthermore, we assume $h(n) \geq 0$, for all nodes n . Therefore, for all goal nodes m^* , $h(m^*) = 0$.

Theorem If h is admissible, then A* search is optimal: i.e., if A* returns goal node m^* , then $g(m^*) \leq g(n^*)$, where n^* is an optimal goal node.

We provide two proofs of this claim, one by contradiction, and a second, by induction. Note that the theorem statement assumes termination; hence, a full proof of correctness requires this theorem together with a proof

that A* indeed terminates. We defer a discussion of termination to the next subsection on completeness. Note also that these proofs apply to trees, not graphs. More work is needed to prove optimality on graphs.

Proof by Contradiction The proof is by contradiction. Suppose A* returns goal node m^* s.t. $g(m^*) > g(n^*)$, where n^* is an optimal goal node. But then it must be the case that there exists node n on the frontier that is on an optimal path for which $f(n) < f(m^*)$, by the following argument:

$$\begin{aligned}
f(n) &= g(n) + h(n) && \text{by the definition of } f \\
&\leq g(n) + h^*(n) && \text{by admissibility} \\
&= g(n^*) + && \text{distance to optimal goal} \\
&< g(m^*) && \text{by assumption} \\
&\leq g(m^*) + h(m^*) && \text{since } h(n) \geq 0, \text{ for all nodes } n \\
&= f(m^*) && \text{by the definition of } f
\end{aligned}$$

But n was not expanded and A* expands nodes in order of f -cost. Contradiction. $\square$

Proof by Induction We establish the following loop invariant of the A* algorithm: there always exists a node n on the frontier s.t. n is on an optimal path and $f(n) \leq g(n^*)$. Given this invariant, if A* returns goal node m^* , then $g(m^*) = g(m^*) + h(m^*) = f(m^*) \leq g(n^*)$.

$$\begin{aligned}
g(m^*) &= g(m^*) + h(m^*) && h(m^*) = 0, \text{ since } m^* \text{ is a goal node} \\
&= f(m^*) && \text{by the definition of } f \\
&\leq g(n^*) && \text{by the invariant}
\end{aligned}$$

Base Case. At a start state s ,

$$\begin{aligned}
f(s) &= g(s) + h(s) && \text{by the definition of } f \\
&= h(s) && g(s) = 0, \text{ since } s \text{ is a start state} \\
&\leq h^*(s) && \text{by admissibility} \\
&\leq g(n^*) && \text{since } s \text{ is on an optimal path}
\end{aligned}$$

Inductive Step. Assume there exists a node n on the frontier s.t. n is on an optimal path and $f(n) \leq g(n^*)$. We must show this invariant continues to hold after the next node is expanded.

Case 1. A* does not expand n . This case is trivial: n remains on the frontier.

Case 2. A* expands n and generates a successor node n' along the same optimal path. It holds that $f(n') \leq g(n^*)$, since

$$\begin{aligned}
f(n') &= g(n') + h(n') && \text{by the definition of } f \\
&\leq g(n') + h^*(n') && \text{by admissibility} \\
&= g(n^*) && \text{since } n' \text{ is on an optimal path}
\end{aligned}$$

$\square$

1.2 Completeness

A* search is complete—meaning it always discovers a goal node, if one exists—assuming admissibility,¹ assuming a locally finite graph (i.e., a finite branching factor b), and the search space does not contain infinitely many nodes n with $f(n) < g(n^*)$ (i.e., an infinite path of finite value), where n^* is an optimal goal. To ensure that this latter property holds, we assume all costs are positive; in particular, there exists $\alpha > 0$ s.t. $c(n, m) \geq \alpha$ for all $n, m \in \mathcal{T}(n)$.

¹in trees, and consistency (see Section ??) in graphs

Theorem A* search is complete: i.e., it always discovers a goal node, if one exists.

Proof Recall A*'s loop invariant, assuming admissibility: there always exists a node l on the frontier s.t. l is on an optimal path and $f(l) \leq g(n^*)$.

Alternatively, consider an arbitrary node m on the frontier. If m is expanded before l , then $f(m) \leq f(l)$. Thus, regardless of which node n (l or m) is expanded next, $f(n) \leq g(n^*)$.

Moreover, for all nodes n , $h(n) \geq 0$ by assumption, so the next node n to be expanded satisfies

$$g(n) \leq g(n) + h(n) = f(n) \leq g(n^*)$$

Now, assume node n is at depth d . Since costs are always positive, $g(n) \geq d\alpha$. But then $d\alpha \leq g(n^*)$; or equivalently, $d \leq g(n^*)/\alpha$, i.e., d is finite. Additionally, since b is also finite, there are only finitely many nodes s.t. $f(n) \leq g(n^*)$, so only finitely many nodes can be expanded before A* necessarily discovers a goal.

2 IDA* Search

Iterative deepening A* (IDA*) is an optimal search algorithm with the performance properties of A*—it is complete and optimal—and the space requirements of DFS—(essentially) linear in depth. The main idea of iterative deepening A* is to repeatedly search in depth-first fashion, over subgraphs with f -cost less than α , less than 2α , less than 3α , and so on, until a goal node is found, where α is a lower bound on the cost between nodes and their successors throughout the search space: i.e., $0 \leq \alpha \leq c(n, m)$, for all $n, m \in \mathcal{T}(n)$.

Recall that the space complexity of ID is $O(bd)$, where d is the depth of the goal node. Similarly, the space complexity of IDA* is $O(bg^*/\alpha)$, where g^* is the optimal cost. The time complexity of IDA*, however, can exceed that of A*. In particular, in search spaces where the f -cost is different at every state, only one additional state is expanded during each iteration. In such a search space, if A* expands N nodes, IDA* expands $1 + \dots + N = O(N^2)$ nodes. The typical solution to this problem is to fix an increment $\beta \geq \alpha$ such that several nodes n have cost $f_i < f(n) \leq f_i + \beta$, where f_i is the i th incremental value of the f -cost. This strategy reduces search time, since the total number of iterations is proportional to $1/\beta \leq 1/\alpha$, and returns solutions that are at worst β -optimal: i.e., if IDA* returns n^* , then $g(n^*) \leq g^* + \beta$.

3 Summary

Criteria	Best- g
Time	$O(b^d)$: BFS, if $g = \text{depth}$
Space	$O(b^d)$: BFS, if $g = \text{depth}$
Completeness	YES, if there do not exist ∞ -many nodes n s.t. $g(n) < g^*$
Optimality	YES, if g is monotonically nondecreasing in depth

Criteria	Best- h
Time	$O(b^d)$: BFS, if $h = \text{depth}$
Space	$O(b^d)$: BFS, if $h = \text{depth}$
Completeness	NO, if nodes are visited in DFS order
Optimality	NO, if nodes are visited in DFS order

Criteria	A*
Time	$O(b^d)$: BFS, if $g = \text{depth}$ and $h = 0$
Space	$O(b^d)$: BFS, if $g = \text{depth}$ and $h = 0$
Completeness	YES, if there do not exist ∞ -many nodes n s.t. $f(n) < f^*$
Optimality	YES, if h is admissible on trees, or consistent on graphs

IDA* SEARCH	
Inputs	search problem $\langle X, S, G, \mathcal{T}, c \rangle$ heuristic function h
Output	(path to) optimal goal node
Initialize	$i = 0$ is the cutoff f -value $O = S$ is the list of open nodes
<pre> while (1) do 1. while (O is not empty) do (a) delete <i>first</i> node $n \in O$ (b) if $n \in G$, return (path to) goal node n (c) for all $m \in \mathcal{T}(n)$ i. compute $h(m)$ ii. $g(m) = g(n) + c(n, m)$ iii. $f(m) = g(m) + h(m)$ iv. if $f(m) \leq i$, insert m in <i>front</i> of O 2. increment i by β, $O = S$ </pre>	

Table 2: Iterative Deepening A*.

Criteria	IDA*
Time	$O(N^2)$, if f -costs differ at all states and A* expands n nodes
Space	$O(bg^*/\beta)$, if f is monotonically nondecreasing in depth and if g^* optimal is the optimal cost
Completeness	YES, if there do not exist ∞ -many nodes n s.t. $f(n) < f^* + \beta$
β -Optimality	YES, if h is admissible on trees, or consistent on graphs

4 Example

A* and IDA* Search The tree depicted in Figure 1 has cost function $f(n) = g(n) + h(n)$. A* search returns the optimal goal node G in this example. Nodes are expanded as follows: A, BCD, ECFD, CFD, GFD, GOAL! Or, if ties are broken otherwise, nodes could be expanded in an alternative order: A, BCD, CEDF, EGDF, GDF, GOAL! Since h is admissible, A* is optimal. IDA* expands nodes as follows, for $\beta = 1$: $f = 0$: A; $f = 1$: AB; $f = 2$: ABECG, GOAL!

5 A* on Graphs

Figure 2 depicts a graph, labelled with cost and heuristic functions. This heuristic is indeed admissible, which is necessary for optimality, but more is needed for A* to return an optimal goal node on graphs.

A* begins by expanding nodes as follows: S_0 , A_2B_4 , C_3B_4 , B_4G_5 . At this point, B is popped off the queue, and its successor C, which has already been visited, must be *reopened*, because it has been rediscovered via a shorter path! Thus, assuming a closed set (of already visited nodes) is maintained, A* continues as C_2G_5 .

But maintaining a closed set and potentially re-opening nodes is not enough to guarantee the optimality of A* on graphs. Nodes already on the priority queue must be promoted if a shorter path to them is discovered.

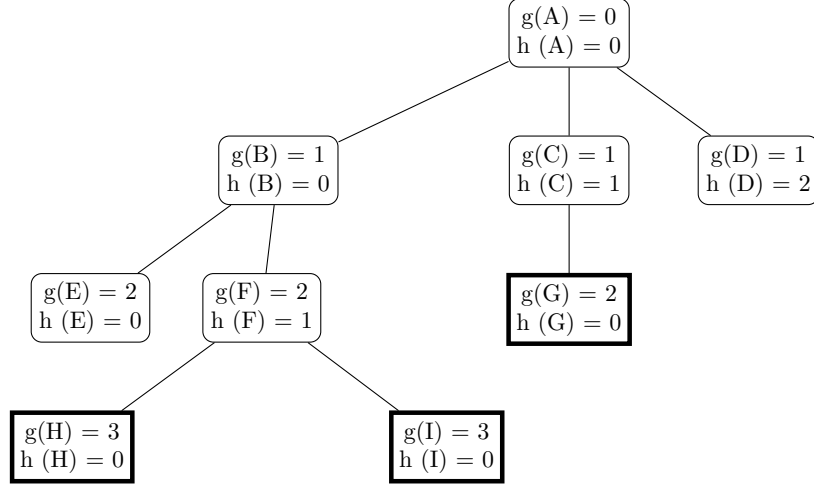


Figure 1: Sample search tree, labeled with costs g and heuristic values h . Boxes indicate goal nodes. A^* search returns the optimal goal node G .

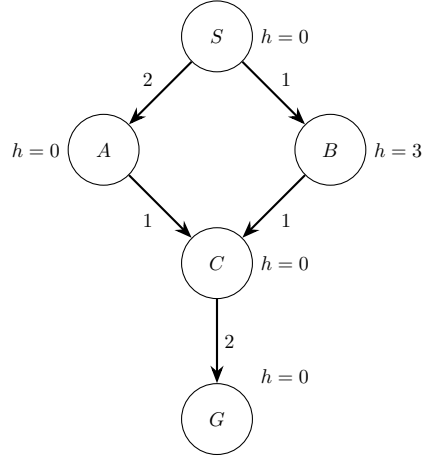


Figure 2: Sample graph, labeled with costs c and heuristic values h . G is the goal.

Continuing, C popped a second time. But at this point, a shorter path to G is discovered. So G , which is already on the priority queue, must be *promoted* from priority 5 to priority 4. The queue is thus G_4 (not G_5) when G is finally popped, and an optimal path to the goal is indeed returned.

Maintaining a closed set and potentially reopening nodes is expensive in terms of memory usage, but can be avoided. The key is to impose a stronger property than admissibility on heuristics, namely consistency.

Definition A heuristic is **consistent** iff $h(n) \leq c(n, m) + h(m)$.

In words, the heuristic cost of a node n and its successor m cannot differ by more than the cost of the edge from n to m : i.e., $h(n) - h(m) \leq c(n, m)$.

Note that consistency is a stronger property than admissibility.

Theorem Consistency implies admissibility.

Proof Assume the heuristic h is consistent. By definition, for every edge $n \rightarrow m$ with cost $c(n, m)$,

$$h(n) \leq c(n, m) + h(m)$$

Consider an arbitrary node n and an *optimal* path from n to a goal node n^* : $n = n_0 \rightarrow n_1 \rightarrow \dots \rightarrow n_k = n^*$.

Now, applying consistency repeatedly yields

$$\begin{aligned} h(n_0) &\leq c(n_0, n_1) + h(n_1) \\ &\leq c(n_0, n_1) + c(n_1, n_2) + h(n_2) \\ &\vdots \\ &\leq \sum_{i=0}^{k-1} c(n_i, n_{i+1}) + h(n_k) \\ &= \sum_{i=0}^{k-1} c(n_i, n_{i+1}) \\ &= h^*(n_0) \end{aligned}$$

The penultimate line follows because $n_k = n^*$ is a goal, which implies that $h(n_k) = 0$. The last line follows because the path was chosen to be optimal, so its total cost equals the true cost of reaching an optimal goal from n_0 , namely $h^*(n_0)$. Since n_0 was arbitrary, $h(n) \leq h^*(n)$, for all nodes n , i.e., h is admissible. $\square$

Although h is admissible for the graph in Figure 2, consistency fails. In particular, $h(B) - h(C) = 3 - 0 = 3$, while $c(B, C) = 1$, and $3 \not\leq 1$.

What is the implication of inconsistency? In our example, the f -value along the path from B to C decreased from $f(B) = 4$ to $f(C) = 2$. This inconsistency is what caused A* to visit A before B. With a consistent heuristic, nodes are visited in non-decreasing f -order.

For h to be consistent in our example, $h(C)$ must equal 2. Let's run through A* again with this new heuristic.

A* begins by expanding nodes as follows: S_0 , A_2B_4 , B_4C_5 . At this point, B is popped off the queue, and its successor C, which is already on the priority queue, must be promoted from priority 5 to priority 4. But unlike before, C does not need to be re-opened—and a (potentially very expensive) closed set need not be maintained! Finally, C_4 is popped off the queue and an optimal path to the goal G is discovered.

Theorem Assuming a consistent heuristic, A* necessarily visits nodes in non-decreasing f -order.

Proof Consider a node n and a successor m .

By consistency,

$$h(n) \leq c(n, m) + h(m)$$

Plugging in $c(m) - c(n)$ for $c(n, m)$ yields

$$h(n) \leq c(m) - c(n) + h(m)$$

Rearranging yields

$$c(n) + h(n) \leq c(m) + h(m)$$

Therefore, $f(n) \leq f(m)$. $\square$

The story does not quite end here, however, because consistency is not something that can be easily verified in advance, as most heuristics are computed on the fly, i.e., as nodes are expanded. There is, however, a local correction that is often implied, namely the **pathmax** equation:

$$h(m) \leftarrow \max\{h(m), h(n) - c(n, m)\}$$

In our running example, as long as B is visited, C's heuristic value can be updated via the pathmax equation to $h(C) \leftarrow \max\{h(C), h(B) - c(B, C)\} = \max\{0, 3 - 1\} = 2$ before C is pushed onto the priority queue.